Session 2.2

Nonlinearity: the Activation, XOR, and What a Network Can Approximate

Why a layer without σ buys nothing, and which functions the family with it can reach



Redistribution prohibited without permission
@Zhigang Feng

Zhigang Feng

What We Are Building, and What We Will Ask of It

- Deck 2.1 left one neuron, $h = \sigma(w^\top x + b)$, and one unexamined piece: the **activation** σ.
- **Question 1.** What is σ, and which ones will you actually meet?
- **Question 2.** Why is it needed at all — what fails in a layer without it, and what does a second layer fix that a wider first layer cannot?
- **Question 3.** How do kinks add up to a curve, and which functions can the family reach?
- Economists already choose flexible functional forms: translog, CES, polynomials, splines.
- **A network is one more flexible form. The difference is that it chooses its own basis.**

© Zhirang Feng · Redistribution prohibited without permission

Session 2.2

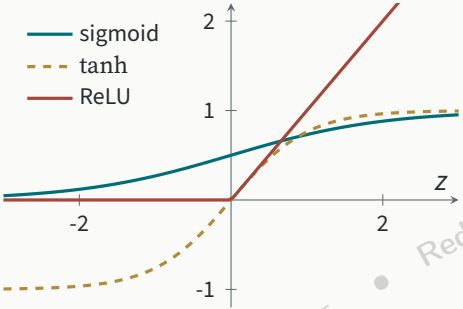
The Activation

The one nonlinear piece, and the three you will meet



© Zhigang Feng Redistribution prohibited without permission

The Activations You Will Actually Meet

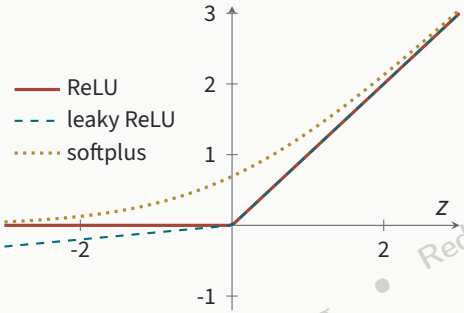


- **Sigmoid**, $1/(1+e^{-z})$, in $(0,1)$; **tanh**, in $(-1,1)$. Both **saturate**: flat in the tails.
- **ReLU**, $\max(0,z)$: zero on one side, the identity on the other. Slope exactly 0 or 1.
- ReLU for hidden layers; sigmoid where an output must be bounded.
- **No activation on a regression output: it would silently bound the prediction.**

Source: Miao & Feng, *AI for Economists*, §9.2, Table 9.2.

© Zhigang Feng
Redistribution prohibited without permission

ReLU Is a Hinge

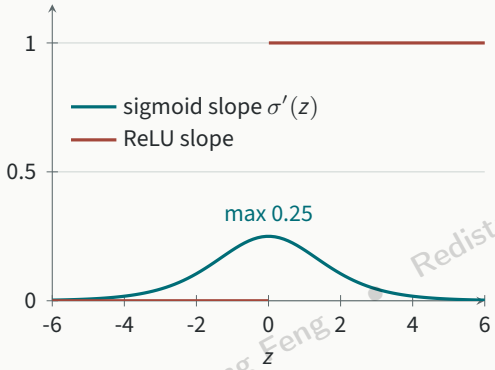


© Zhigang Feng
Redistribution prohibited without permission

- $\max(0, z)$ is a call option's payoff, and an occasionally binding constraint: inactive, then linear.
- **Leaky ReLU**: a small slope on the flat side, so a unit that switched off can return.
- **Softplus**, $\log(1 + e^z)$, rounds the corner, for models that need second derivatives.
- **Each hidden ReLU unit contributes one hinge. A network is a sum of hinges.**

Source: Miao & Feng, *AI for Economists*, §9.2, Fig. 9.4.

Saturation, and Where the Vanishing Gradient Begins



- The sigmoid's slope is $\sigma(z)(1-\sigma(z))$, never above 0.25.
- Training sends an error signal backwards through the layers (Deck 2.4, autograd), multiplied by this slope once per layer: at best 0.25^L.
- $L=3$: 0.016. $L=5$: 0.00098. $L=7$: 6×10^{-5}. The early layers barely move: the **vanishing gradient**.
- **ReLU's slope is 1 on its active side, so the signal survives depth. That is why it is the default.**

Source: Miao & Feng, *AI for Economists*, §9.2.

© Zhigang Feng Redistribution prohibited without permission

Session 2.2

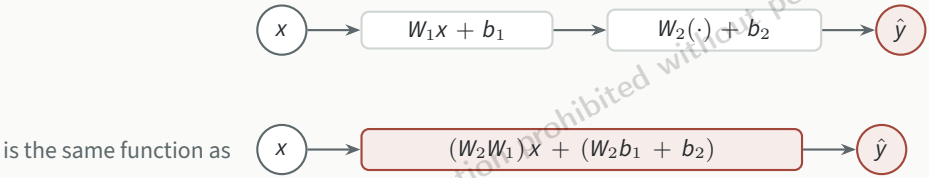
Why One Layer Is Not Enough

What collapses without σ, and what a hidden layer represents exactly



© Zhigang Feng
Redistribution prohibited without permission

Linear Stacks Stay Linear

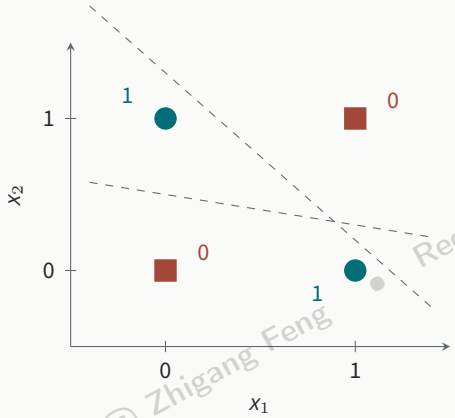


© Zhigang Feng — Redistribution prohibited without permission

- Remove the activation and compose two layers: $W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$.
- That is one linear map. A hundred linear layers are still one regression line.
- **Every bit of a network's flexibility comes from the activation between the linear maps.**

Source: Miao & Feng, *AI for Economists*, §9.1.

What One Neuron Cannot Do

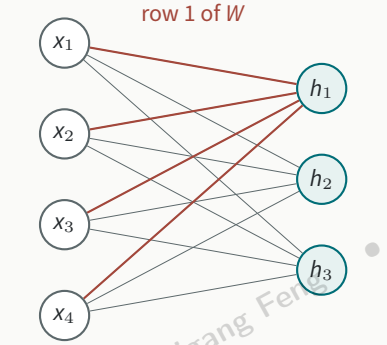


Source: M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969.

- **XOR**, exclusive or: the output is 1 when exactly one of two binary inputs is 1.
- The two 1s sit on one diagonal, the two 0s on the other. No straight line separates them: try the dashed ones.
- Minsky and Papert (1969) made this the standard objection to single-layer networks.
- **The cure is a second layer. That is the rest of this section.**

© Zhigang Feng
Redistribution prohibited without permission

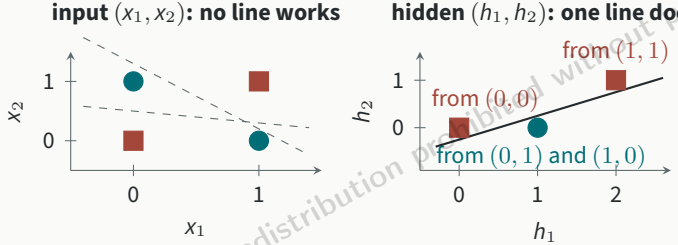
A Layer Is Many Neurons Reading the Same Input



- Put m neurons side by side. Each has its own weights and bias; all read the same x.
- Stack the weight vectors as rows of a matrix W ($m \times n$): $h = \sigma(Wx + b)$, with σ applied entry by entry.
- Here W is 3×4: 12 weights and 3 biases, 15 parameters.
- **A layer is a system of m regressions sharing their regressors.**

© Zhigang Feng Redistribution prohibited without permission

The Same Four Points, Moved by the Hidden Layer

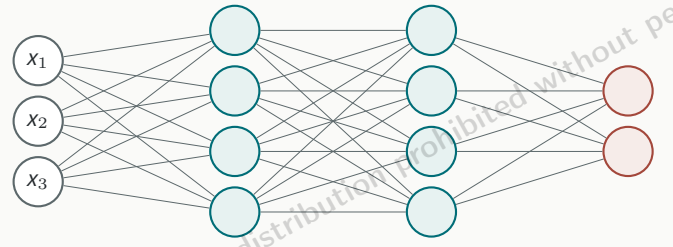


- **The same four points, drawn twice.** Same marks, same colours; only the axes change.
- Write $s = x_1 + x_2$. Then $h_1 = \max(0, s)$ and $h_2 = \max(0, s - 1)$: $(0,0) \mapsto (0,0)$, both $(0,1)$ and $(1,0) \mapsto (1,0)$, $(1,1) \mapsto (2,1)$.
- The drawn line is the output $\hat{y} = h_1 - 2h_2$: $0,\ 1,\ 1,\ 0$, *exact*, not approximate. **The hidden layer did not fit a curve — it moved the points until one line was enough.**

Source: Miao & Feng, *AI for Economists*, §9.2; Goodfellow, Bengio, Courville, *Deep Learning*, 2016, §6.1.

© Zhiqiang Feng · Redistribution prohibited without permission

The Object in One Line

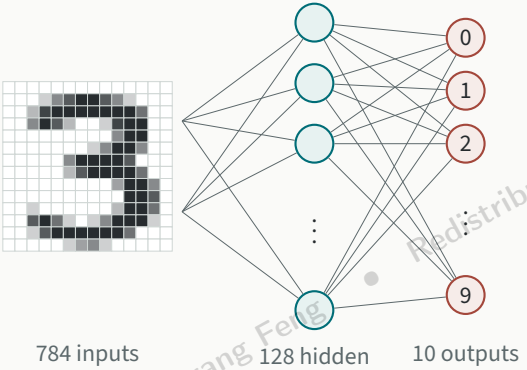


© Shigeki Fujii. Redistribution prohibited without permission

- A **feedforward network** (multilayer perceptron, MLP) composes layers: $f_\theta(x) = W_3\,\sigma\big(W_2\,\sigma(W_1 x + b_1) + b_2\big) + b_3$.
- θ collects every W_ℓ, b_ℓ. **Width**: units per layer. **Depth**: number of layers.
- **A network is a parametric family: alternate a linear map with a fixed nonlinearity.**

Source: Miao & Feng, *AI for Economists*, §9.1, eq. (9.6).

A Network for Digits: 784 → Hidden → 10



- Layer 1: 784×128 weights $+ 128$ biases $= 100{,}480$ parameters.
- Layer 2: $128 \times 10 + 10 = 1{,}290$. Total: **101,770**.
- Trained as in Deck 2.3, it classifies 97 to 98% of the 10,000 test images correctly.
- **More parameters than observations, and it still generalises. Deck 2.6 asks why.**

© Zhigang Feng. Redistribution prohibited without permission

Source: Accuracy: the 784–128–10 example of the TensorFlow beginner tutorial; the same one-hidden-layer design in LeCun et al. (1998), 300 units: 95.3%.

Session 2.2

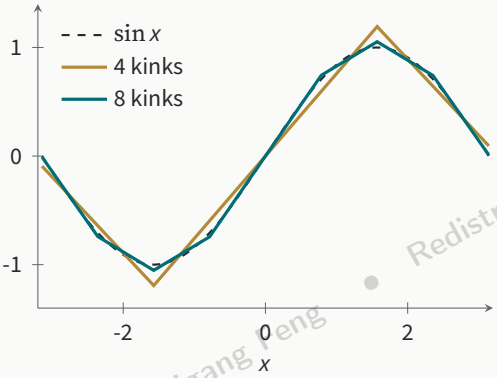
How a Network Bends a Straight Line

Approximating $\sin x$ out of hinges, and letting the hinges move



© Zhigang Feng Redistribution prohibited without permission

The Construction: an Approximation Built out of Kinks

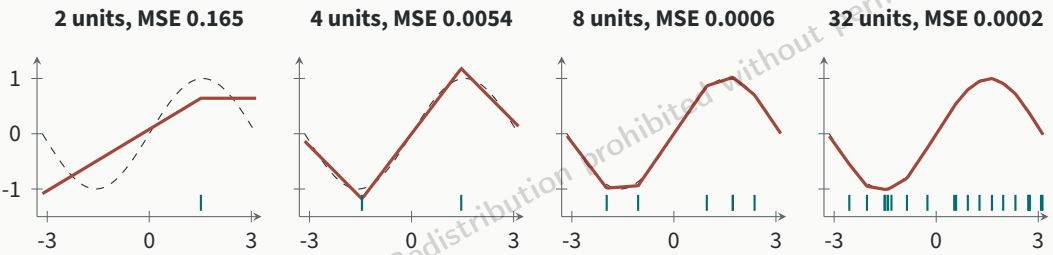


- One hidden ReLU layer, one input: $\hat{y}(x) = a_0 + \sum_{i=1}^{N} a_i \max(0,\, x - b_i)$.
- The shifts b_i are the **kinks**, where the slope may change; the a_i set the slope on each interval.
- Fix the b_i, evenly spaced, and $\hat{y}$ is linear in the a_i: ordinary least squares on a piecewise-linear basis.
- **Mean squared error 0.0059 with 4 kinks, 0.0003 with 8. There is no network here yet.**

Source: Session 2 lab notebook, block B; least squares on 121 points of $\sin x$.

© Zhigang Feng
Redistribution prohibited without permission

The Turn: Let the Kinks Choose Where to Go

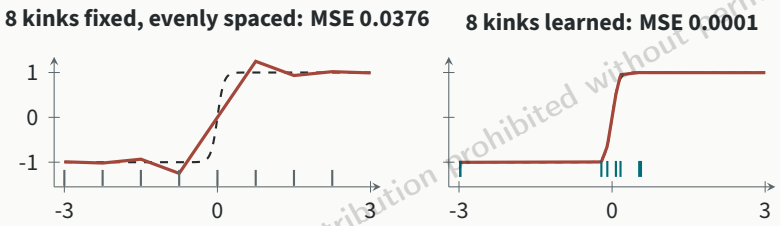


Redistribution prohibited without permission

- Train the b_i together with the a_i: a one-hidden-layer network. The teal ticks mark the learned kinks, and they are not evenly spaced.
- At 8 units fixed kinks by least squares (0.0003) beat training (0.0006); at 4 units training (0.0054) beat fixed (0.0059). A search, not a solve.
- **The basis is no longer chosen in advance. The approximant decides where to bend.**

Source: Session 2 lab notebook, block B; Adam, 3,000 steps, one seed.

Where Adaptivity Pays

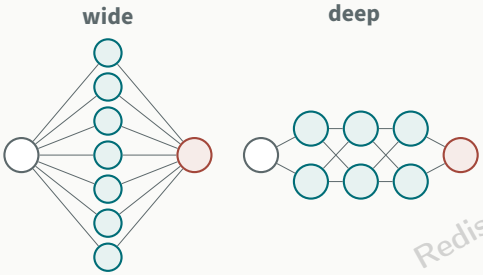


© Zhigang Feng Redistribution prohibited without permission

- $\tanh(8x)$: all the curvature near zero. Eight units either way.
- Six of the eight learned kinks crowd into an eighth of the domain; the fit is **396 times** better. On $\sin x$, whose curvature is spread evenly, training lost.
- A fixed basis is linear in its parameters: a vector space, a closed-form solve, a rate quotable in advance. Width-*m* networks are none of these.
- **A solve becomes a search. What that buys is priced next.**

Source: Adam, 6,000 steps, one seed, against least squares on the same eight kinks. Session 2 lab notebook, block B.

Width and Depth: Two Axes



- Width adds basis functions; depth composes them.
- Cost: 20 inputs, three hidden layers of 256 units:
  $5{,}376 + 65{,}792 + 65{,}792 + 257 = 137{,}217$
  parameters.
- Some functions need exponentially many units in two layers, polynomially many in three.
- **Depth pays when the target is a composition: data $\to$ beliefs $\to$ choice; households $\to$ aggregates.**

Source: R. Eldan and O. Shamir, "The power of depth for feedforward neural networks", COLT 2016; Miao & Feng, §9.1–9.2.

© Zhigang Feng
Redistribution prohibited without permission

Session 2.2

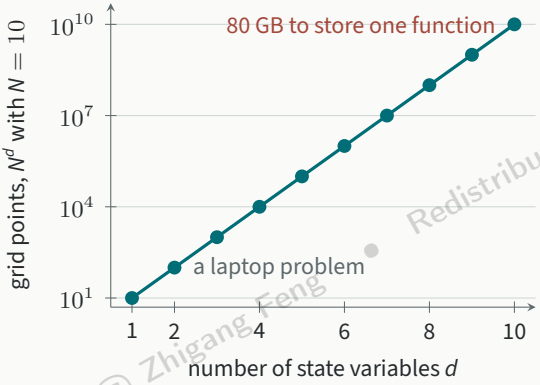
What the Theory Says

Existence, rates, and what neither one promises



© Zhigang Feng Redistribution prohibited without permission

The Curse of Dimensionality, in Arithmetic



- A grid with N points per state variable needs N^d points in d dimensions.
- With $N = 10$: 100 points at $d = 2$, a million at $d = 6$, ten billion at $d = 10$, which is 80 GB for one policy function.
- Wealth, income, age, housing and two aggregate shocks already make $d = 6$.
- **A method whose cost grows with the number of examples, not with N^d, changes what can be solved.**

© Zhigang Feng
Redistribution prohibited without permission

Universal Approximation, on a Compact Domain

- A **compact domain** is closed and bounded: $[0,1]^d$, or a capital range $[k_{\min}, k_{\max}]$.
- **Universal approximation**: for any continuous f on a compact domain and any $\varepsilon > 0$ there is a one-hidden-layer network $\hat{f}(x) = \sum_{j=1}^{m} v_j \, \sigma(w_j^{\top} x + b_j) + c$ with $\sup_x |f(x) - \hat{f}(x)| < \varepsilon$.
- Cybenko (1989) for sigmoids; Hornik (1991) for any bounded non-constant activation; Leshno et al. (1993) for any that is not a polynomial.
- Polynomials have the same property (Stone–Weierstrass), so universality does not distinguish a network.
- **The theorem says a good network exists on the domain. It says nothing outside the domain.**

© Zhiyang Feng · Redistribution prohibited without permission

Source: Cybenko, *Math. Control Signals Syst.* 2, 1989; Hornik, *Neural Networks* 4, 1991; Leshno, Lin, Pinkus, Schocken, *Neural Networks* 6, 1993.

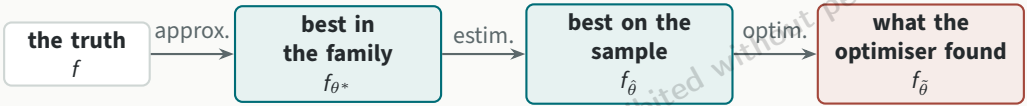
Four Things the Theorem Does Not Say

- **Not how many units.** "There exists m", and m may be astronomically large.
- **Not how to find the weights.** Good weights exist; finding them is a non-convex search that promises only a stationary point.
- **Not how much data.** Approximating a known function is different from estimating an unknown one from a finite sample.
- **Not that it will generalise.** Fitting the sample and fitting the function are different achievements.
- **Finding the weights** is the binding one. With 8 kinks, least squares reached mean squared error 0.0003; training the same 8 units reached 0.0006. A better member of the family existed and the optimiser did not find it.
- **Existence is a statement about the family, not about the network you trained.**

Source: Miao & Feng, *AI for Economists*, §9.1.

© Zhaonan Feng · Redistribution prohibited without permission

Three Errors: Approximation, Estimation, Optimization



- **Approximation error**: the family cannot reach f. Falls with width and depth. The theorems live here.
- **Estimation error**: a finite sample picks the wrong member. Falls with data, rises with flexibility.
- **Optimization error**: the algorithm stops short of the sample optimum. Deck 2.3.
- **Only the sum is observed, on held-out data. A bigger network lowers the first error and can raise the other two.**

Source: L. Bottou and O. Bousquet, “The tradeoffs of large scale learning”, NeurIPS 2007.

© Zhiheng Huang Redistribution prohibited without permission

Barron's Theorem, and the Class It Holds For

- Barron (1993): with m sigmoidal hidden units, on a ball of radius r,

$$\int_{B_r} \left(f - \hat{f}_m\right)^2 d\mu \;\le\; \frac{(2rC_f)^2}{m}, \qquad C_f = \int \|\omega\|\,|\tilde{f}(\omega)|\,d\omega,$$

  where $\tilde{f}$ is the Fourier transform of f.
- The **Barron class** is the set of functions with finite C_f: little weight at high frequencies.
- Double the units and the squared error halves, and the input dimension d is absent from the exponent.
- But C_f is an integral over $\mathbb{R}^d$. For many functions it grows with d; for others it is infinite.
- **The dimension did not vanish. It moved into the constant and into the narrowness of the class.**

Source: A. Barron, "Universal approximation bounds for superpositions of a sigmoidal function", *IEEE Trans. Inf. Theory* 39, 1993.

© Zhengling Qi · Redistribution prohibited without permission

The Rates, Side by Side

Functions	Approximator, size m	Best possible	Source
k times smooth	any continuous method	$\gtrsim m^{-k/d}$ (error)	DeVore et al. (1989)
Barron class	any basis fixed in advance	$\gtrsim m^{-2/d}$ (squared)	Barron (1993), Thm. 6
Barron class	one hidden layer	$\leq C/m$ (squared)	Barron (1993), Prop. 1
compositional	matching deep network	exponent free of d	Poggio et al. (2017)

© Zhiguo Feng · Redistribution prohibited without permission

- Rows 1–2 are the curse of dimensionality. Rows 3–4 escape it by assuming something about f.
- **The gap is between a basis fixed before seeing f and a basis adapted to f.**

Source: DeVore, Howard, Micchelli, *Manuscripta Math.* 63, 1989; Poggio et al., *Int. J. Autom. Comput.* 14, 2017.

Takeaways, and What Deck 2.3 Has to Supply

- **Without an activation, depth buys nothing.** With ReLU, each hidden unit is one hinge and a network is a sum of hinges.
- **A hidden layer changes the coordinates** until a line is enough; XOR shows this exactly, not approximately.
- **Fix the kinks and it is least squares; free them and the approximant chooses where to bend.**
- **Universal approximation is existence on a compact domain**: not size, not a recipe, not data, not generalisation.
- **Barron's rate has no d in the exponent**; the dimension lives in the constant and the class.
- **Every promise here was "a good network exists". Deck 2.3 is how to find one, and how to know when to stop.**

© Zhigang Feng · Redistribution prohibited without permission